

Website Tracking Code Notes

Scope: Project code explanation for developer and performance marketer. GTM container ID is intentionally not included.

1. Layout.jsx Role

Layout.jsx tracking code load nahi karta. Is file ka kaam common page structure render karna hai.

```
function Layout() {  
  return (  
    <div>  
      <Header />  
      <Outlet />  
      <Footer />  
    </div>  
  );  
}
```

Header har page par visible hai. Outlet ke andar current route ka page render hota hai.

Footer har page ke bottom par visible hai.

2. App Routes

App.jsx me React Router routes define hain. Checkout aur Success payment flow ke liye use hote hain.

```
<Route path="/" element={<Layout />}>  
  <Route index element={<Home />} />  
  <Route path="checkout" element={<Checkout />} />  
  <Route path="success" element={<Success />} />  
</Route>
```

User cart se Proceed Item click karta hai to /checkout par jata hai. Fake Pay Now click hone par /success par navigate hota hai.

3. GTM/dataLayer Helper Function

Project me ek common helper banaya gaya hai jo event object ko browser ke dataLayer me push karta hai.

```
export const pushToDataLayer = (data: object) => {  
  window.dataLayer = window.dataLayer || [];  
  window.dataLayer.push(data);  
};
```

Is helper ko har tracking action me import karke use kiya gaya hai. Isse code repeat kam hota hai aur event push ka standard way milta hai.

4. Redux Toolkit Cart Flow

Cart state Redux Toolkit slice me maintain hoti hai. Cart localStorage se load hota hai aur update hone par localStorage me save hota hai.

```
const storedCart = JSON.parse(localStorage.getItem("cart")) || {  
  items: [],  
  totalQuantity: 0,  
  totalPrice: 0,  
};
```

addItem reducer cart item add/increase karta hai, totalQuantity aur totalPrice update karta hai.

```

addItem(state, action) {
  const newItem = action.payload;
  const existingItem = state.items.find((item) => item.id === newItem.id);
  const itemName = newItem.name || newItem.FirstP;

  state.totalQuantity++;
  const itemPrice = parseFloat(newItem.price);

  if (!existingItem) {
    state.items.push({
      id: newItem.id,
      name: itemName,
      FirstP: itemName,
      price: itemPrice,
      quantity: 1,
      total: itemPrice,
      watch: newItem.watch,
      img: newItem.img,
      secondP: newItem.secondP
    });
    state.totalPrice += itemPrice;
  } else {
    existingItem.name = existingItem.name || itemName;
    existingItem.FirstP = existingItem.FirstP || itemName;
    existingItem.quantity++;
    existingItem.total += itemPrice;
    state.totalPrice += itemPrice;
  }

  saveToLocalStorage(state);
}

```

5. item_name Undefined Fix

Products me name kabhi name field me tha aur kabhi FirstP field me. Isliye item_name undefined aa sakta tha. Fix me dono fields ka fallback use hua.

```
const itemName = newItem.name || newItem.FirstP;
```

Events me fallback pattern use hua:

```
item.name || item.FirstP || "Unknown Product"
```

Agar browser localStorage me old item pehle se undefined ke saath saved hai, to test ke liye cart clear karke item dobara add karna chahiye.

6. add_to_cart Event

Product ADD button click hone par Redux dispatch ke baad add_to_cart event dataLayer me push hota hai.

```

const handleAddToCart = (item) => {
  dispatch(addItem(item));

  pushToDataLayer({
    event: "add_to_cart",
    ecommerce: {
      currency: "INR",
      value: Number(item.price),
      items: [
        {
          item_id: item.id,
          item_name: item.name || item.FirstP,
          price: Number(item.price),

```

```

        quantity: 1,
      },
    ],
  },
});
};

```

7. view_cart Event

Cart drawer open hone par view_cart event push hota hai. Ye cart state se items map karta hai.

```

pushToDataLayer({
  event: "view_cart",
  ecommerce: {
    currency: "INR",
    value: cart.totalPrice,
    items: cart.items.map((item) => ({
      item_id: item.id,
      item_name: getItemName(item),
      price: Number(item.price),
      quantity: item.quantity,
    })),
  },
});

```

getItemName helper saved item name, FirstP, item_name ya previous dataLayer item se name resolve karta hai. Last fallback Item ID hai, so undefined avoid hota hai.

8. begin_checkout Event

Cart page me Proceed Item button click hone par begin_checkout event push hota hai, phir user /checkout route par navigate hota hai.

```

const handleCheckout = () => {
  pushToDataLayer({
    event: "begin_checkout",
    ecommerce: {
      currency: "INR",
      value: cart.totalPrice,
      items: cart.items.map((item) => ({
        item_id: item.id,
        item_name: getItemName(item),
        price: item.price,
        quantity: item.quantity,
      })),
    },
  });

  navigate("/checkout");
};

```

9. purchase Event

Checkout page me Fake Pay Now button click hone par purchase event push hota hai, phir user /success route par jata hai.

```

const handlePayment = () => {
  pushToDataLayer({
    event: "purchase",
    ecommerce: {
      transaction_id: "ORDER_" + Date.now(),
      currency: "INR",
      value: cart.totalPrice,

```

```

        items: cart.items.map((item) => ({
            item_id: item.id,
            item_name: item.name || item.FirstP || "Unknown Product",
            price: item.price,
            quantity: item.quantity,
        })),
    },
});

console.log("purchase fired");
navigate("/success");
};

```

10. Success Page

Purchase event ke baad Success component render hota hai. Ye confirmation UI hai.

```

const Success = () => {
    return (
        <div style={{ padding: "40px" }}>
            <h1>Payment Successful</h1>
            <p>Purchase Event Sent</p>
        </div>
    );
};

```

11. Ecommerce Object Meaning

- event: event name jise GTM trigger sunta hai, e.g. add_to_cart or purchase.
- ecommerce.currency: currency code, yahan INR.
- ecommerce.value: product/cart total value.
- ecommerce.transaction_id: purchase order id.
- ecommerce.items: product list array.
- item_id: product id.
- item_name: product name.
- price: product price.
- quantity: product quantity.

12. Live Tracking Explanation

Tag Assistant Connect sirf preview/debug ke liye hota hai. Real users ko connect karne ki zarurat nahi hoti.

Live tracking flow: website loads script, user action hota hai, React handler pushToDataLayer call karta hai, GTM published trigger event pakadta hai, phir Ads/Analytics tag fire hota hai.

13. UTM Status

Is project me UTM reader/pusher code add nahi kiya gaya. Current implementation ecommerce dataLayer events ke liye hai.

Example UTM URL:
 ?utm_source=google&utm_medium=cpc&utm_campaign=sale

UTM tab add karte hain jab campaign parameters ko custom reporting ke liye code se read karke dataLayer me push karna ho.

14. GTM vs GA4 vs UTM Comparison

GTM: Tag manager. Website par tags fire/manage karta hai. Iska kaam tags deploy karna hai.

GA4: Analytics platform. Users, sessions, ecommerce events aur reports analyze karta hai.

UTM: URL campaign parameters. Source, medium, campaign identify karte hain.

15. Final Developer Note

Code side me Redux Toolkit cart state, React Router navigation, aur dataLayer event helper connected hain. Marketing tags ko GTM UI me configure/publish karna separate step hai.